



PL/SQL ASSIGNMENT_2

Name: MUTANGANA Joseph ID: 29061 ASSIGNMENT2 REPORT

Course Code and Name: INSY 8311 – Database Development with PL/SQL

TASK A: SQL Recap

A1. (a): Query

```
SET SERVEROUTPUT ON
select d.department_name, d.location, nvl(count(e.employee_id),0) as
number_of_employees,
nvl(sum(e.salary),0) as total_salary_bill,
round(nvl(avg(e.salary),0), 2) as average_salary, (select
nvl(count(c.course_id),0) from courses c where c.department = d.department_name)
as total_courses
from departments d
left join
employees e on e.department = d.department_name
group by d.department_name, d.location
order by nvl(sum(e.salary),0) desc ;
```

Output

DEPARTMENT_NAME	LOCATION	NUMBER_OF_EMPLOYEES	TOTAL_SALARY_BILL	AVERAGE_SALARY	TOTAL_COURSES
1 Computer Science	Building A Floor 3	60	4328000	72133.33	10
2 Business	Building C - Floor 1	36	2068000	57444.44	1
3 Engineering	Building D - Floor4	24	1755600	73150	0
4 Psychology	Building E - Floor 2	20	1340000	67000	1
5 Mathematics	Building B - Floor 2	16	1320000	82500	2
6 English	Building F - Floor 1	0	0	0	1

A1. (b): Query

```
SET SERVEROUTPUT ON
select d.department_name, d.budget, (select sum(e.salary) from employees e where
e.department = d.department_name) as total_salary_bill,
(select sum(e.salary) from employees e where e.department = d.department_name)-
d.budget as Amount_over_budget
from departments d
group by d.department_name, d.budget
```



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
having (select sum(e.salary) from employees e where e.department =  
d.department_name) > d.budget ;
```

Output

	DEPARTMENT_NAME	BUDGET	TOTAL_SALARY_BILL	AMOUNT_OVER_BUDGET
1	Computer Science	750000	2164000	1414000
2	Business	550000	1034000	484000
3	Mathematics	450000	660000	210000
4	Psychology	400000	670000	270000
5	Engineering	800000	877800	77800

TASK B: Variables, %TYPE, %ROWTYPE and DBMS_OUTPUT

B1: Query

```
set SERVEROUTPUT on  
  
declare  
v_c_name varchar2(100) := 'PL/SQL Programming';  
v_cr number(1) := 3;  
v_date date := date '2024-09-01';  
is_heavy boolean := TRUE;  
v_check varchar2(5);  
  
begin  
dbms_output.put_line('Course: ' || v_c_name);  
dbms_output.put_line('Credits: ' || v_cr);  
dbms_output.put_line('Semester Start: ' || v_date);  
  
if v_cr >= 3 then  
is_heavy := TRUE;  
v_check := 'TRUE';  
else  
is_heavy := FALSE;  
v_check := 'FALSE';  
end if;  
dbms_output.put_line('Heavy Course: ' || v_check);
```



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
end;  
/
```

Output

Lecturer x

```
Course: PL/SQL Programming  
Credits: 3  
Semester Start: 01-SEP-24  
Heavy Course: TRUE
```

B2: Query

```
set SERVEROUTPUT on  
  
declare  
v_course_id number :=110;  
v_course_name courses.course_name%TYPE;  
v_cr courses.credits%TYPE;  
v_instructor courses.instructor%TYPE;  
v_max courses.max_students%TYPE;  
  
begin  
select course_name,credits, instructor, max_students into  
v_course_name,v_cr, v_instructor, v_max  
from courses where course_id = v_course_id;  
  
dbms_output.put_line('Course Name: '||v_course_name);  
dbms_output.put_line('Credits: '||v_cr);  
dbms_output.put_line('Instructor: '||v_instructor);  
dbms_output.put_line('Max_students: '||v_max);  
if v_max <=25 then  
dbms_output.put_line('Limited seats available!');  
end if;  
end;  
/
```

Output



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Lecturer ✕

Course Name: Machine Learning
Credits: 4
Instructor: Dr.
Thomas
Max_students: 20
Limited seats available!

B3: Query

```
set SERVEROUTPUT on

declare
v_employee employees%ROWTYPE;
v_hire_year    NUMBER;
v_years_worked NUMBER;

begin
select * into v_employee from employees where employee_id = 22;
v_hire_year := extract(year from v_employee.hire_date);
v_years_worked := trunc(months_between(sysdate, v_employee.hire_date) / 12);
dbms_output.put_line('Full Name: '||v_employee.first_name||' '||
v_employee.last_name);
dbms_output.put_line('Department: '||v_employee.department);
dbms_output.put_line('Salary: '||v_employee.salary);
dbms_output.put_line('Hire Year: '||v_hire_year);
dbms_output.put_line('Years Worked: '||v_years_worked);
end;
/
```

Output



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Lecturer x

Full Name: Amy Professor
Department: Computer Science
Salary: 102000
Hire Year: 2014
Years Worked: 12

TASK C: IF / ELSIF / ELSE Conditions

C1: Query

```
declare

v_employee_id number :=1;
v_commission_pct number;
v_salary number;
v_potential_earning number;

begin
select commission_pct, salary into v_commission_pct, v_salary
from employees where employee_id = v_employee_id;

if v_commission_pct is not NULL and v_salary >80000 then
dbms_output.put_line('High-value manager with commission');
elsif v_commission_pct is not NULL and v_salary <=80000 then
dbms_output.put_line('Manager with commission, standard
tier');
elsif v_commission_pct is NULL and v_salary >90000 then
dbms_output.put_line('Senior employee, consider adding a
commission incentive');
else
dbms_output.put_line('Standard employee profile');
end if;
v_potential_earning := v_salary + (v_salary * NVL(v_commission_pct, 0));
dbms_output.put_line('Total potential earnings: '|| v_potential_earning);

EXCEPTION
when no_data_found then
dbms_output.put_line('No data found matching the '|| v_employee_id);

when others then
```



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
dbms_output.put_line('Unexpected error');
```

```
end;
```

```
/
```

Output

```
High-value manager with commission  
Total potential earnings: 89250
```

C2: Query

```
declare  
  
v_course_id number :=103;  
v_c_name varchar2(50);  
v_enrolled number;  
v_max_students number;  
v_availability_status varchar(100);  
  
begin  
select count(*) into v_enrolled from enrollments  
where course_id = v_course_id;  
  
select course_name, max_students into v_c_name, v_max_students from courses  
where course_id = v_course_id;  
  
dbms_output.put_line('Course Name: '|| v_c_name);  
dbms_output.put_line('Maximum Capacity: '|| v_max_students);  
dbms_output.put_line('Current Enrollment: '|| v_enrolled);  
if v_enrolled < (v_max_students * 0.5) then  
v_availability_status := 'Open, plenty of seats available';  
  
elsif v_enrolled BETWEEN (v_max_students * 0.5) and (v_max_students * 0.8) then  
v_availability_status := 'Filling up, limited seats';  
  
elsif v_enrolled > (v_max_students * 0.8) and v_enrolled < v_max_students then
```



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
v_availability_status := 'Almost full, enroll soon';

elsif v_enrolled >= v_max_students then
v_availability_status := 'FULL, enrollment closed';
end if;
dbms_output.put_line('Availability Status: ' || v_availability_status);

EXCEPTION
when no_data_found then
dbms_output.put_line('No data found matching the ' || v_course_id);

when others then
dbms_output.put_line('Unexpected error');

end;
/
```

Output

```
Course Name: PL/SQL Programming
Maximum Capacity: 25
Current Enrollment: 10
Availability Status: Open, plenty of seats available
```

C3: Query

```
declare

v_student_id number := 1020;
v_dob date;
v_years number;
v_age number;
v_enrollment_date date;
v_enrollment_year number;

begin
select date_of_birth, enrollment_date into v_dob, v_enrollment_date
from students
```



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
where student_id = v_student_id;

v_years := trunc(months_between(sysdate, v_dob) / 12);
v_enrollment_year := extract(year from v_enrollment_date);

if v_years <18 then
dbms_output.put_line('Minor student, parental consent required');
elsif v_years >=18 and v_years <=25 then
dbms_output.put_line('Traditional student age');
elsif v_years > 25 then
dbms_output.put_line('Mature student, eligible for the evening program');
end if;

if v_enrollment_year = 2023 then
dbms_output.put_line('First-year cohort (2023)');
else
dbms_output.put_line('Joined in ['||v_enrollment_year||']');
end if;
EXCEPTION
when no_data_found then
dbms_output.put_line('No data found matching the '|| v_student_id);

when others then
dbms_output.put_line('Unexpected error');

end;
/
```

Output

```
Traditional student age
Joined in [2024]
```

Section D: CASE Statements and Expressions

D1: Query



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
SET SERVEROUTPUT ON

declare

v_enrollment_id number :=5;
v_grade varchar2(25);
v_stored_grade varchar2(25);
v_score number;

begin
select score, grade into v_score, v_stored_grade from enrollments
where enrollment_id = v_enrollment_id;

case
    when v_score is null then
        v_grade := 'Not yet graded';
    when v_score >= 90 then
        v_grade := 'A';
    when v_score >= 80 then
        v_grade := 'B';
    when v_score >= 70 then
        v_grade := 'C';
    when v_score >= 60 then
        v_grade := 'D';
    else
        v_grade := 'F';
end case;

if v_grade = v_stored_grade then
dbms_output.put_line('Grade match: YES');
else
dbms_output.put_line(
    'Grade match: NO (stored ' || v_stored_grade ||
    ', computed ' || v_grade || ')')
);
end if;

EXCEPTION

when no_data_found then
dbms_output.put_line('No data found matching the ' || v_enrollment_id);
```



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
when others then
dbms_output.put_line('Unexpected error');

end;
/
```

Output

```
Grade match: YES
```

D2: Query

```
SET SERVEROUTPUT ON

declare

v_department varchar2(100) := 'Computer Science';
v_dep_id number :=10;
v_building_code varchar2(50);
v_budget number;
v_budget_status varchar2(50);

begin
select department_name, budget into v_department, v_budget from departments
where department_id = v_dep_id;

case
    when v_department = 'Computer Science' then
        v_building_code := 'BLD-CS | ICT Wing';

    when v_department = 'Mathematics' then
        v_building_code := 'BLD-MT | Science Wing';
```



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
when v_department = 'Business' then
    v_building_code := 'BLD-BS | Commerce Wing';
when v_department = 'Engineering' then
    v_building_code := 'BLD-EN | Technical Wing';
when v_department = 'Psychology' then
    v_building_code := 'BLD-PS | Humanities Wing';
else
    v_building_code := 'BLD-GN | General Wing';
end case;

v_budget_status := CASE
    WHEN v_budget > 600000 THEN 'Well Funded'
    WHEN v_budget >= 400000 THEN 'Adequately Funded'
    ELSE 'Underfunded'
END;

dbms_output.put_line('Building Code: ' || v_building_code);
dbms_output.put_line('Budget Status: ' || v_budget_status);

EXCEPTION

when no_data_found then
dbms_output.put_line('No data found matching the ' || v_department);
when others then
dbms_output.put_line('Unexpected error');

end;
/
```

Output

```
Building Code: BLD-CS | ICT Wing
Budget Status: Well Funded
```

D3: Query

```
SET SERVEROUTPUT ON
```



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
declare

v_emp_id number :=14;
v_name varchar2(50);
v_salary number;
v_job_title varchar2(50);
v_rank number;
v_rank_title varchar2(50);

begin
select first_name||' '|| last_name, salary, job_title into v_name, v_salary,
v_job_title from employees
where employee_id = v_emp_id;

case
    when v_job_title = 'Full Professor' then
        v_rank :=1;
        v_rank_title := 'Senior Academic';

        when v_job_title = 'Associate Professor' then
            v_rank :=2;
            v_rank_title := 'Mid Academic';
        when v_job_title = 'Assistant Professor' then
            v_rank :=3;
            v_rank_title := 'Junior Academic';
        when v_job_title like '%Developer%' or v_job_title like '%Engineer%' then
            v_rank :=4;
            v_rank_title := 'Technical Staff';
        when v_job_title like '%Admin%' or v_job_title like '%Coordinator%' then
            v_rank :=5;
            v_rank_title := 'Administrative Staff';
        else
            v_rank :=6;
            v_rank_title := 'General Staff';
end case;

dbms_output.put_line('Name: '|| v_name);
dbms_output.put_line('Job Title: '|| v_job_title);
```



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
dbms_output.put_line('Rank Number: ' || v_rank);
dbms_output.put_line('Rank Title: ' || v_rank_title);
dbms_output.put_line('Salary: ' || v_salary);

if v_rank <= 2 and v_salary < 100000 then
dbms_output.put_line('Promotion eligible');
else
dbms_output.put_line('Not eligible for promotion yet');
end if;

EXCEPTION

when no_data_found then
dbms_output.put_line('No data found matching the ' || v_emp_id);
when others then
dbms_output.put_line('Unexpected error');

end;
/
```

Output

```
Name: Nina Professor
Job Title: Assistant Professor
Rank Number: 3
Rank Title: Junior Academic
Salary: 88000
Not eligible for promotion yet
```

Section E: Loops

E1: Query

```
SET SERVEROUTPUT ON
DECLARE
```



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
v_tuition number :=150000;
v_c_name VARCHAR2(100);
v_c_cr NUMBER;
v_c_id number :=101;
v_total_cr number :=0;
v_frw number;
v_total_frw number :=0;
v_avg_cost number;

begin

loop

select course_name, credits into v_c_name, v_c_cr
from courses where course_id = v_c_id;
v_c_id := v_c_id +1;
v_total_cr := v_total_cr + v_c_cr;
v_frw := v_c_cr * v_tuition;
v_total_frw :=v_total_frw + v_frw;

dbms_output.put_line(rpad('Added: '||v_c_name,20)||' ('||v_c_cr||') |' || rpad('
Total: '||v_total_cr,10)||' credits | RWF '||v_frw );
exit when v_total_cr >= 15;

end loop;
v_avg_cost := v_total_frw / v_total_cr;
dbms_output.put_line('Total Cost: '||v_total_frw);
dbms_output.put_line('Average cost per credit: '||v_avg_cost);
end;
/
```

Output

```
Added: Introduction (3) | Total: 3 credits | RWF 450000
Added: Database Mana (4) | Total: 7 credits | RWF 600000
Added: PL/SQL Progra (3) | Total: 10 credits | RWF 450000
Added: Data Structur (4) | Total: 14 credits | RWF 600000
Added: Web Developme (3) | Total: 17 credits | RWF 450000
Total Cost: 2550000
Average cost per credit: 150000
```



E2: Query

```
SET SERVEROUTPUT ON

DECLARE
depart varchar(100) := 'Computer Science';
total_salary number :=0;
total_emp number :=0;
higher_earners number :=0;
low_earners number :=0;
highest_paid number :=0;
highest_paid_name varchar2(100);
v_name employees.first_name%TYPE;
v_last_name employees.last_name%TYPE;
v_salary employees.salary%TYPE;
avg_salary number;
cursor emp_list is
select first_name, last_name,salary
from employees where department=depart;

begin

open emp_list;
fetch emp_list into v_name,v_last_name, v_salary;
while emp_list%found loop
total_emp :=total_emp + 1;
total_salary :=total_salary + v_salary;
if v_salary > highest_paid then
highest_paid := v_salary;
highest_paid_name := v_name||' '||v_last_name;
end if;
if v_salary > 75000 then
higher_earners := higher_earners + 1;
elsif v_salary < 65000 then
low_earners := low_earners + 1;
end if;
fetch emp_list into v_name,v_last_name, v_salary;
end loop;
close emp_list;
avg_salary := total_salary / total_emp;
dbms_output.put_line('==== '||depart||' Salary Report ====');
dbms_output.put_line('Total Employees: '||total_emp);
```



Output

E3: Query

```
set serveroutput on

declare

v_id number;
v_name varchar2(100);
v_gpa number;
v_label varchar2(100);
v_load varchar2(100);
v_count number;

v_students_printed number := 0;
v_summa number := 0;
v_magna number := 0;
v_cum number := 0;
```




ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
v_satisfactory number := 0;
v_probation number := 0;

begin

for v_id in 1001..1010 loop

select first_name||' '||last_name,gpa into v_name,v_gpa
from students where student_id = v_id;

select count(student_id) as total into v_count
from enrollments where student_id = v_id;

v_students_printed := v_students_printed + 1;

case

when v_gpa >= 3.9 then
    v_label := 'Summa Cum Laude';
    v_summa := v_summa + 1;

when v_gpa >= 3.7 then
    v_label := 'Magna Cum Laude';
    v_magna := v_magna + 1;

when v_gpa >= 3.5 then
    v_label := 'Cum Laude';
    v_cum := v_cum + 1;

when v_gpa >= 3.0 then
    v_label := 'Satisfactory';
    v_satisfactory := v_satisfactory + 1;

else
    v_label := 'Probation';
    v_probation := v_probation + 1;

end case;

case

when v_count >= 4 then
    v_load := 'Full Load';
```



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
when v_count >= 2 then
    v_load := 'Normal Load';

else
    v_load := 'Under-enrolled';

end case;

dbms_output.put_line(
    rpad('ID: ' || v_id, 10) || '|' ||
    rpad(v_name, 20) || '|' ||
    rpad('GPA: ' || v_gpa, 10) || '|' ||
    rpad(v_label, 20) || '|' ||
    rpad(v_load, 16)
);

end loop;

dbms_output.put_line('Number of students printed: ' || v_students_printed);
dbms_output.put_line('Summa Cum Laude: ' || v_summa);
dbms_output.put_line('Magna Cum Laude: ' || v_magna);
dbms_output.put_line('Cum Laude: ' || v_cum);
dbms_output.put_line('Satisfactory: ' || v_satisfactory);
dbms_output.put_line('Probation: ' || v_probation);

end;
/
```

Output



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

ID: 1001	Alice Johnson	GPA: 3.8	Magna Cum Laude	Full Load
ID: 1002	Bob Smith	GPA: 3.2	Satisfactory	Normal Load
ID: 1003	Carol White	GPA: 3.5	Cum Laude	Normal Load
ID: 1004	David Green	GPA: 2.9	Probation	Normal Load
ID: 1005	Emma Davis	GPA: 3.9	Summa Cum Laude	Normal Load
ID: 1006	Frank Miller	GPA: 3.4	Satisfactory	Normal Load
ID: 1007	Grace Wilson	GPA: 3.7	Magna Cum Laude	Normal Load
ID: 1008	Henry Moore	GPA: 3.1	Satisfactory	Normal Load
ID: 1009	Ivy Taylor	GPA: 3.6	Cum Laude	Normal Load
ID: 1010	Jack Anderson	GPA: 2.8	Probation	Normal Load

Number of students printed: 10
Summa Cum Laude: 1
Magna Cum Laude: 2
Cum Laude: 2
Satisfactory: 3
Probation: 2

Section F: Combined Challenge

F1: Query

```
set serveroutput on

declare

emp employees%rowtype;
emp_id number :=5;
hired_year number;
years_of_service number;
service_class varchar(50);
pay_grade number(1);
depart varchar2(100);
count_colleagues number :=0;
highest_salary number;
count_more_earners number :=0;
proposed_salary number;
raise_status varchar2(50);
```



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
begin
select * into emp from employees where employee_id = emp_id;

years_of_service :=(months_between(sysdate, emp.hire_date) / 12);

case
when years_of_service >5 then
service_class := 'Senior Staff';
when years_of_service >=2 then
service_class := 'Confirmed';
else
service_class := 'Probationary';
end case;

-- Pay grades: 1 = 100k+, 2 = 90k-99,999, 3 = 80k-89,999,
--              4 = 70k-79,999, 5 = below 70k

case
when emp.salary >= 100000 then
pay_grade :=1;
when emp.salary >= 90000 then
pay_grade :=2;
when emp.salary >= 80000 then
pay_grade :=3;
when emp.salary >= 70000 then
pay_grade :=4;
else
pay_grade :=5;
end case;

highest_salary :=emp.salary;
for new_emp in (select * from employees where department = emp.department and
employee_id != emp.employee_id) loop

count_colleagues :=count_colleagues +1;
if new_emp.salary > highest_salary then
highest_salary := new_emp.salary;
end if;

if new_emp.salary > emp.salary then
count_more_earners := count_more_earners + 1;
end if;
```



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

```
end loop;
dbms_output.put_line('=====');
dbms_output.put_line('  EMPLOYEE PROFILE REPORT
  AUCA Staff Records System');
dbms_output.put_line('=====');
dbms_output.put_line('Employee: '||emp.first_name||' '||emp.last_name||' (ID:
'||emp.employee_id||)');
dbms_output.put_line('Department: '||emp.department);
dbms_output.put_line('Job Title: '||emp.job_title);
dbms_output.put_line('Hire Date: '||emp.hire_date||' | Years of Service:
'||to_char(years_of_service, 'fm0.0'));
dbms_output.put_line('Service Status: '||service_class);
dbms_output.put_line('-----');
dbms_output.put_line('Salary: RWF '||TO_CHAR(emp.salary, '999,999')||' | Pay
Grade: '||pay_grade);
if years_of_service > 3 and emp.salary <85000 then
proposed_salary := emp.salary * 1.10;
raise_status:='Raise recommended';
dbms_output.put_line('Raise Status: '||raise_status);
dbms_output.put_line('Proposed Salary: RWF '||TO_CHAR(proposed_salary,
'999,999'));
else
raise_status:='No raise due';
dbms_output.put_line('Raise Status: '||raise_status);
end if;
dbms_output.put_line('-----');
dbms_output.put_line('Dept. Colleagues: '||count_colleagues);
dbms_output.put_line('Dept Highest Salary: RWF '||TO_CHAR(highest_salary,
'999,999'));
dbms_output.put_line('Colleagues Earning More: '||count_more_earners);
dbms_output.put_line('=====');

end;
/
```

Output



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

AUCA Staff Records System

=====

Employee: Tom Engineer (ID: 5)
Department: Computer Science
Job Title: Software Engineer
Hire Date: 15-SEP-19 | Years of Service: 7.1
Service Status: Senior Staff

Salary: RWF 80,000 | Pay Grade: 3
Raise Status: Raise recommended
Proposed Salary: RWF 88,000

Dept. Colleagues: 29
Dept Highest Salary: RWF 102,000
Colleagues Earning More: 4

=====

Section G: SQL

G1: Query

```
WITH stats AS (  
    SELECT  
        COUNT(*) AS total_employees,  
        SUM(salary) AS total_salary,  
        AVG(salary) AS average_salary,  
        SUM(CASE WHEN salary > 75000 THEN 1 ELSE 0 END) AS higher_earners,  
        SUM(CASE WHEN salary < 65000 THEN 1 ELSE 0 END) AS lower_earners,  
        MAX(salary) AS highest_paid  
    FROM employees  
    WHERE department = 'Computer Science'  
),  
top_employee AS (  
    SELECT  
        first_name || ' ' || last_name AS highest_paid_name,  
        salary  
    FROM employees  
    WHERE department = 'Computer Science'  
    ORDER BY salary DESC  
    FETCH FIRST 1 ROW ONLY  
)
```



```
FROM stats
CROSS JOIN top_employee;
```

[illegible][illegible]

Approach I would use



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

So between SQL and PL/SQL, I would use PL/SQL approach, because it is more flexible for me, because with PL/SQL I can declare separate variables and later i do query to select data I want into variables, and later I make decision based on those variables.